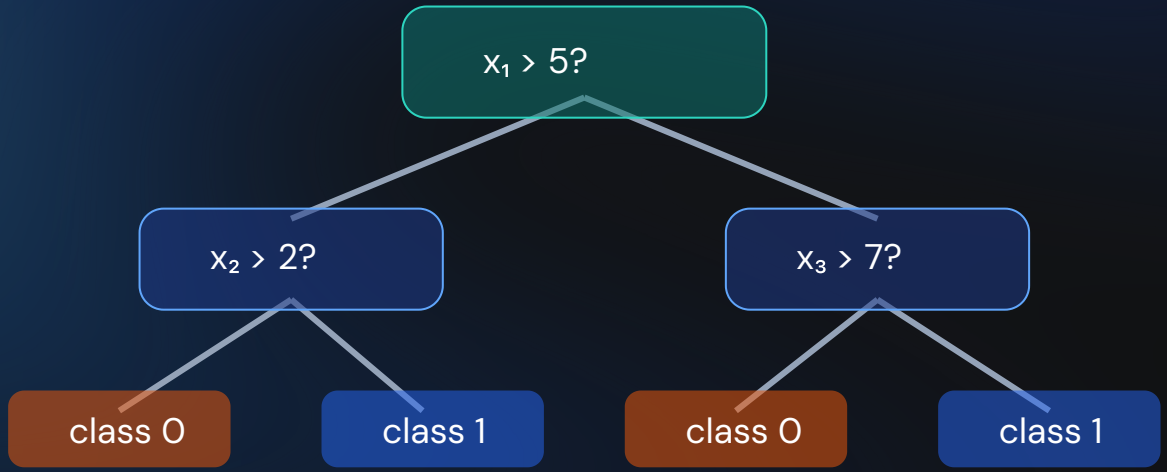
Decision Trees

Classification via a sequence of questions

Supervised Learning · Classification

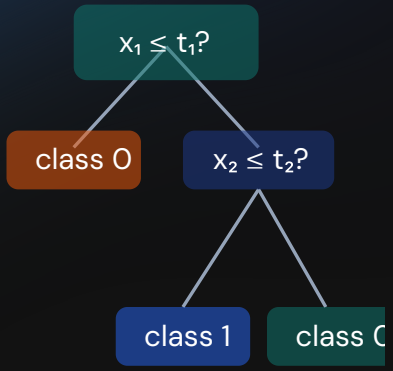


A Tree Is Also a Partition of Space

- Internal nodes test one feature against a threshold
- Follow matching branches from root to leaf
- Leaves hold class predictions
- Binary splits carve feature space into axis-aligned rectangles

```
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier

X, y = make_classification(
    n_samples=200, n_features=2,
    n_redundant=0, random_state=0)
tree = DecisionTreeClassifier(
    max_depth=3, random_state=0)
tree.fit(X, y)
```

feature space

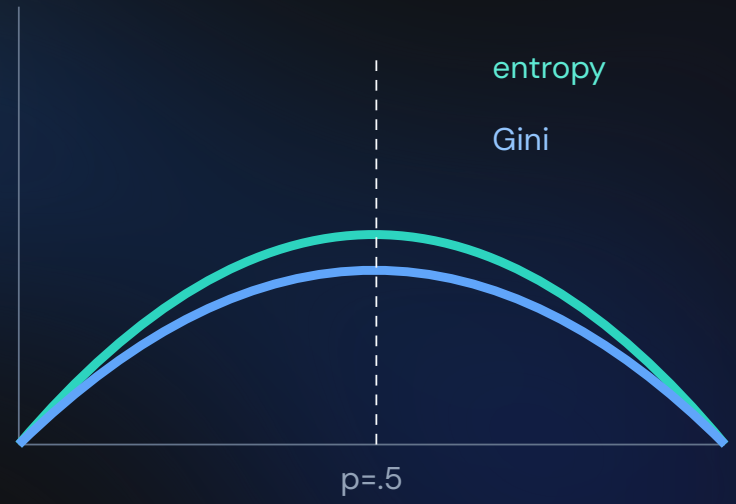


How Pure Is a Node?

$$\text{Gini} = 1 - \sum_{k=1}^{K} p_k^2$$

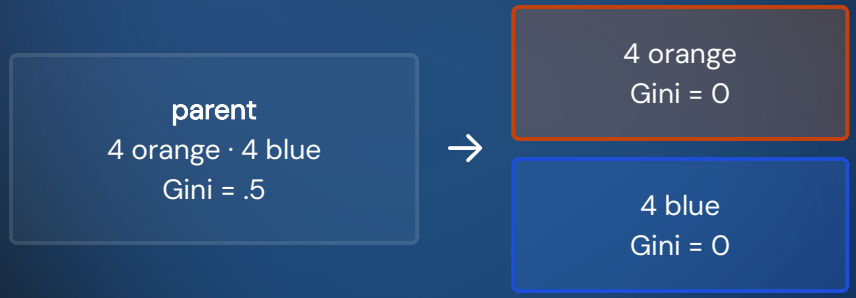
$$\text{Entropy} = -\sum_{k=1}^{K} p_k \log_2 p_k$$

```python
import numpy as np

def gini(p):
    return 1 - np.sum(np.asarray(p)**2)

def entropy(p):
    p = np.asarray(p); p = p[p > 0]
    return -np.sum(p * np.log2(p))

assert gini([1, 0]) == entropy([1, 0]) == 0
```



Information Gain Picks the Split

$$IG = I(parent) - \left[\frac{n_L}{n} I(L) + \frac{n_R}{n} I(R)\right]$$




```
IG = .5 - [.5(0) + .5(0)] = .5
```

```python
def information_gain(parent, left, right):
    def node_gini(labels):
        _, counts = np.unique(labels,
                              return_counts=True)
        return gini(counts / len(labels))
    n = len(parent)
    child = (len(left)/n)*node_gini(left)
    child += (len(right)/n)*node_gini(right)
    return node_gini(parent) - child

p = np.array([0]*4 + [1]*4)
assert information_gain(p, p[:4], p[4:]) == .5
```

A Worked Example With an Imperfect Split

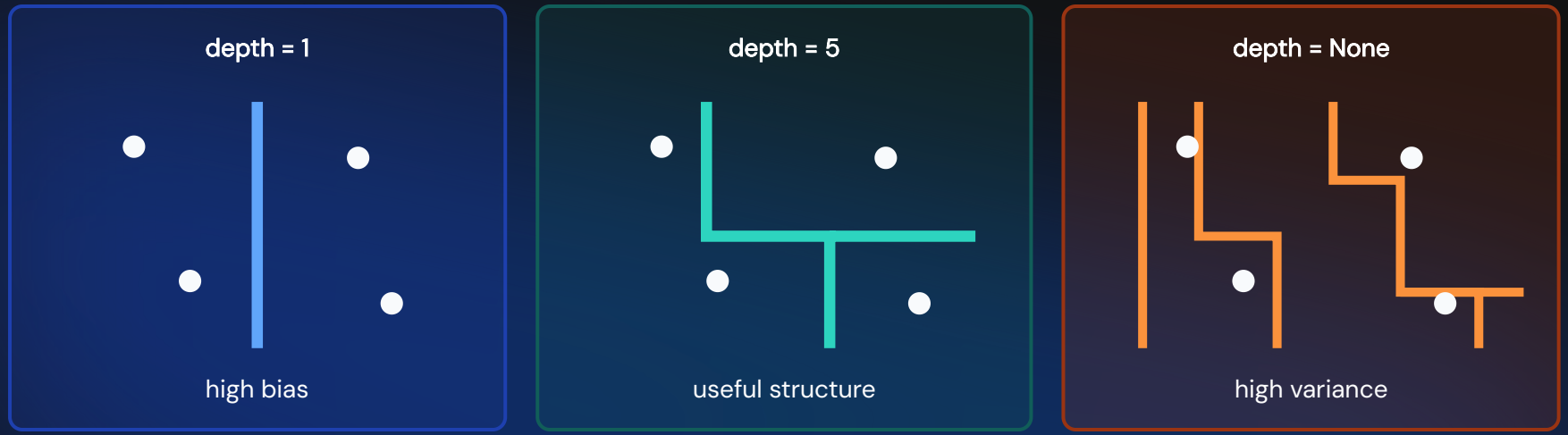
Parent: 10 examples, 6 positive · 4 negative

$$\text{Gini}(parent) = 1 - (0.6^2 + 0.4^2) = 0.48$$

- Left child (5 examples): 4 pos · 1 neg → Gini $= 1 - (0.8^2 + 0.2^2) = 0.32$
- Right child (5 examples): 2 pos · 3 neg → Gini $= 1 - (0.4^2 + 0.6^2) = 0.48$
- Weighted child impurity $= \frac{5}{10}(0.32) + \frac{5}{10}(0.48) = 0.40$
- $IG = 0.48 - 0.40 = 0.08$

```python
def gini_from_counts(pos, neg):
    n = pos + neg
    p_pos, p_neg = pos / n, neg / n
    return 1 - (p_pos**2 + p_neg**2)

parent = gini_from_counts(6, 4)          # 0.48
left, right = gini_from_counts(4, 1), gini_from_counts(2,
child = (5/10) * left + (5/10) * right   # 0.40
ig = parent - child                      # 0.08
assert round(ig, 2) == 0.08
```

Unconstrained Trees Overfit



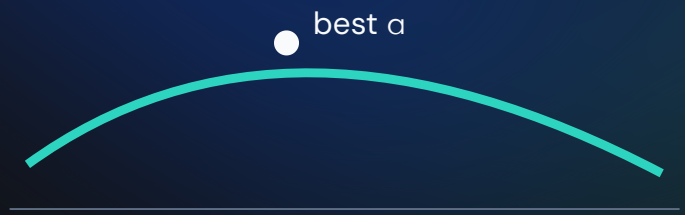

```python
from sklearn.model_selection import cross_val_score

for depth in [1, 3, 5, None]:
    tree = DecisionTreeClassifier(
        max_depth=depth, random_state=0)
    score = cross_val_score(tree, X, y, cv=5).mean()
    print(depth, score)
```

Capacity knobs: `max_depth`, `min_samples_split`, and `min_samples_leaf`.

Pruning: Fit, Then Simplify

$$R_\alpha(T) = R(T) + \alpha|T|$$

- **Pre-pruning:** stop early with depth or sample limits
- **Post-pruning:** grow first, remove weak subtrees later
- `ccp_alpha` controls cost-complexity pruning
- Choose the strength by cross-validation

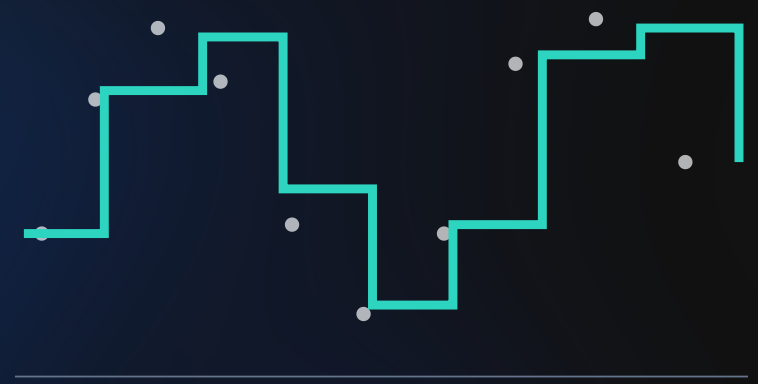



```python
full = DecisionTreeClassifier(random_state=0)
path = full.cost_complexity_pruning_path(X, y)

scores = []
for alpha in path.ccp_alphas:
    model = DecisionTreeClassifier(
        random_state=0, ccp_alpha=alpha)
    scores.append(cross_val_score(
        model, X, y, cv=5).mean())

best_alpha = path.ccp_alphas[np.argmax(scores)]
print(best_alpha)
```

Trees Also Regress

- `DecisionTreeRegressor` predicts the mean target in each leaf
- Splits reduce within-node variance or MSE
- Predictions are piecewise constant—always a staircase
- The same overfitting and pruning controls apply

```
from sklearn.tree import DecisionTreeRegressor

rng = np.random.default_rng(0)
Xr = np.linspace(0, 10, 100)[:, None]
yr = np.sin(Xr[:, 0]) + rng.normal(0, .1, 100)
model = DecisionTreeRegressor(
    max_depth=4, random_state=0)
model.fit(Xr, yr)
assert np.isfinite(model.predict(Xr)).all()
```



Four Classification Strategies

Logistic
discriminative optimization

k-NN
instance-based geometry

Naive Bayes
generative probability

Trees
interpretable rules

Splits greedily maximize impurity reduction; depth limits and pruning control variance.

Next: Model Evaluation · Later: Ensemble Methods build many trees.